



Meteo Sicilia

Stazione meteorologica IoT e applicazione web per il monitoraggio del meteo in tempo reale

Sistema integrato hardware–software: una stazione basata su Raspberry Pi raccoglie i dati ambientali dai sensori e li trasmette a un backend che li distribuisce, in tempo reale, a un'interfaccia web responsiva. Per la città sede della stazione i dati provengono direttamente dai sensori e da un modello di intelligenza artificiale per la previsione; per gli altri comuni della Sicilia si utilizzano dati meteorologici professionali.

Indice

1. **1.** Introduzione e obiettivi
2. **2.** Visione d'insieme e architettura del sistema
3. **3.** La stazione meteorologica (Raspberry Pi) e i sensori
4. **4.** Il backend: FastAPI, MQTT, database e tempo reale
5. **5.** Il frontend: React, TypeScript e Tailwind CSS
6. **6.** Gestione e integrazione dei dati — il caso Palermo
7. **7.** Il grafico orario interattivo in SVG
8. **8.** I «Dettagli meteo»
9. **9.** Design, layout e adattamento responsive
10. **10.** Identità visiva: logo e palette
11. **12.** Scelte tecniche e motivazioni
12. **13.** Problemi affrontati e soluzioni
13. **14.** Sviluppi futuri
14. **15.** Conclusioni

1. Introduzione e obiettivi

Meteo Sicilia è una piattaforma web per la consultazione del meteo in tempo reale, pensata e realizzata attorno a una stazione meteorologica fisica costruita con un **Raspberry Pi**. L'idea di fondo è semplice ma ambiziosa: non limitarsi a riportare previsioni generiche prese da un servizio esterno, ma **misurare davvero** le condizioni ambientali con sensori reali e mostrarle, istante per istante, in un'interfaccia moderna e curata.

Gli obiettivi del progetto sono stati:

- **Acquisire dati reali** (temperatura, umidità, pressione, vento, luce, qualità dell'aria, monossido di carbonio) da una stazione fisica e renderli disponibili online.
- **Distribuire i dati in tempo reale** al sito, in modo che i valori si aggiornino senza dover ricaricare la pagina.
- **Integrare una previsione** per le ore successive prodotta da un modello di **intelligenza artificiale** sviluppato internamente.
- **Coprire l'intera Sicilia**: per i comuni privi di stazione fisica, attingere a dati meteorologici professionali e gratuiti.
- **Offrire un'esperienza d'uso eccellente** su computer, tablet e smartphone, con grafici interattivi e schede di dettaglio leggibili.

Il risultato è un sistema completo e coerente in cui l'elettronica, la programmazione lato server e lo sviluppo web si incontrano: un esempio concreto di applicazione **IoT** (Internet of Things) dal sensore fino allo schermo.

2. Visione d'insieme e architettura del sistema

Il sistema è organizzato in tre strati che comunicano tra loro. Il dato nasce nei **sensori**, viene raccolto e conservato dal **backend** e infine raggiunge l'utente attraverso il **frontend**. Lo schema seguente riassume il flusso completo delle informazioni.

Stazione fisica Raspberry Pi Sensori ambientali + fotocamera misura i dati reali Backend FastAPI (Python) Database SQLite Stream SSE in tempo reale riceve e conserva Frontend React + TypeScript Interfaccia web PC / tablet / telefono mostra all'utente MQTT HTTP / SSE Modello AI previsione ore successive Open-Meteo dati altri comuni + previsioni

2.1 I tre strati

- **Stazione (livello fisico)**. Il Raspberry Pi legge i sensori e invia i valori al backend tramite il protocollo **MQTT**, lo standard più diffuso per l'Internet of Things.
- **Backend (livello applicativo)**. Un server scritto in **Python** con il framework **FastAPI** riceve i messaggi MQTT, li salva in un database e li ripubblica verso il sito in tempo reale.
- **Frontend (livello di presentazione)**. Un'applicazione web in **React** mostra i dati con grafici, schede e una mappa di città, adattandosi a qualsiasi dispositivo.

A questi si affiancano due **fonti dati esterne/intelligenti**: il **modello di intelligenza artificiale**, che prevede l'andamento delle ore successive per la città con stazione, e **Open-Meteo**, un servizio meteorologico professionale e gratuito usato per tutti gli altri comuni siciliani e per le previsioni a sette giorni.

3. La stazione meteorologica (Raspberry Pi) e i sensori

Il cuore fisico del progetto è una stazione costruita attorno a un **Raspberry Pi**, un piccolo computer a basso costo che gestisce i sensori e si occupa di trasmettere le misurazioni. La stazione è collocata nella città sede del progetto (**Palermo**) e rileva un insieme ricco di grandezze ambientali, comprese alcune legate alla **qualità dell'aria**, che i comuni servizi meteo non forniscono.

3.1 Le grandezze misurate

Grandezza	Campo dati	Descrizione e utilizzo
Temperatura	temperatura	Temperatura dell'aria in gradi Celsius; valore principale mostrato nell'interfaccia.
Umidità	umidita	Umidità relativa percentuale; usata anche per calcolare la temperatura percepita e il punto di rugiada.
Pressione	pressione	Pressione atmosferica in hPa; il suo andamento aiuta a intuire i cambi di tempo.
Vento	vento	Velocità del vento in km/h; convertita anche nella scala Beaufort con descrizione testuale.
Luce	luce	Intensità luminosa / irraggiamento solare in percentuale.
Qualità dell'aria	qualita_aria	Indice sintetico (0% ottima → 100% pessima); tradotto in etichetta e colore (Ottima/Buona/Moderata/Scadente).
Monossido di carbonio	monossido_carb	Livello di CO rilevato da sensore di gas; indicatore di sicurezza ambientale.
Immagine	file_path	Fotografia del cielo acquisita dalla fotocamera, salvata e associata all'istante di rilevamento.
Marca temporale	timestamp	Data e ora della lettura; chiave primaria che ordina cronologicamente i dati.

Coerenza con l'hardware reale. I campi dati elencati corrispondono esattamente ai sensori effettivamente montati sulla stazione. L'applicazione è stata costruita rispettando questa struttura: non mostra grandezze inventate e non presuppone sensori non presenti. Questa fedeltà tra software e hardware è stata un vincolo progettuale fondamentale.

3.2 Dalla misura al messaggio

Ad ogni ciclo di lettura il Raspberry Pi compone un messaggio con tutti i valori e lo pubblica su un **broker MQTT** (software Mosquitto) sul canale `stazione1/sensori`; le immagini viaggiano su un canale dedicato `stazione1/immagini`. Il backend, iscritto a questi canali, riceve i messaggi e li registra. Questo disaccoppiamento — la stazione «pubblica» e il server «si iscrive» — rende il sistema robusto e facilmente estendibile a più stazioni future.

4. Il backend: FastAPI, MQTT, database e tempo reale

Il backend è il programma che fa da ponte tra la stazione e il sito. È scritto in **Python** con **FastAPI**, un framework moderno, veloce e *asincrono*, particolarmente adatto a gestire molte connessioni contemporanee e flussi di dati continui. Le sue responsabilità sono tre: **ricevere** i dati dalla stazione, **conservarli** e **distribuirli** al frontend.

4.1 Ricezione e archiviazione

All'avvio, il backend apre una connessione MQTT verso il broker e si mette in ascolto. Ogni messaggio ricevuto viene salvato in un database **SQLite** (gestito in modo asincrono con la libreria `aiosqlite`) nella tabella `dataset`, le cui colonne ricalcano esattamente le grandezze dei sensori. Le immagini vengono scritte su disco e collegate al rispettivo istante. SQLite è stato scelto perché è leggero, non richiede un server separato ed è perfetto per una stazione singola.

4.2 Distribuzione in tempo reale (SSE)

Per aggiornare il sito senza ricaricare la pagina si utilizza la tecnologia **SSE** (Server-Sent Events): il browser apre un canale permanente verso il backend e, ogni volta che arriva una nuova lettura dalla stazione, il server «spinge» un segnale ai client connessi, che aggiornano i valori a schermo. È la stessa logica delle notifiche in tempo reale delle app moderne, ma realizzata con uno standard web semplice ed efficiente.

4.3 I punti di accesso (endpoint)

Endpoint	Funzione
<code>/last_read</code>	Restituisce l'ultima lettura registrata dai sensori.
<code>/sse</code>	Canale in tempo reale: notifica i client a ogni nuova misurazione.
<code>/storico/oggi</code>	Tutte le letture odierne della stazione, usate dal grafico e dai dettagli di Palermo.
<code>/previsione/oggi</code>	Previsione oraria del modello AI per le ore restanti della giornata (predisposto, in attesa dell'integrazione del modello).

Perché il CORS. Durante lo sviluppo il sito gira su un indirizzo diverso da quello del backend. Senza un'esplicita autorizzazione il browser, per sicurezza, bloccherebbe queste chiamate «tra origini diverse». Abbiamo quindi aggiunto il middleware **CORS** che concede al frontend il permesso di interrogare il backend. Non si tratta di «dare internet al server», ma di una regola di sicurezza del browser da configurare correttamente.

Lavoro additivo e rispettoso. La parte di backend che dialoga con i sensori (connessione MQTT, salvataggio nel database, schema della tabella) è stata mantenuta intatta. I nostri interventi — abilitazione del CORS e aggiunta dei due endpoint per Palermo — sono stati **puramente aggiuntivi**, senza modificare la logica esistente né i dati provenienti dalla stazione.

5. Il frontend: React, TypeScript e Tailwind CSS

Il frontend è il sito vero e proprio, ciò che l'utente vede e con cui interagisce. È costruito con tecnologie professionali e largamente diffuse nello sviluppo web moderno.

5.1 Lo stack tecnologico

Tecnologia	Ruolo	Perché è stata scelta
React	Libreria per costruire l'interfaccia a componenti	Permette di scomporre il sito in «mattoncini» riutilizzabili e di aggiornare a schermo solo ciò che cambia.
TypeScript	Linguaggio (JavaScript con i tipi)	Aggiunge controlli automatici che individuano molti errori prima ancora di eseguire il codice.
Vite	Strumento di sviluppo e compilazione	Avvio quasi istantaneo e ricarica immediata delle modifiche durante lo sviluppo.
Tailwind CSS	Sistema di stile	Consente di impaginare in modo rapido e coerente, ideale per il design responsive.
shadcn/ui	Componenti d'interfaccia pronti	Pulsanti, schede e finestre di ricerca già curati ed accessibili, da personalizzare.
React Router	Navigazione tra pagine	Gestisce la home geolocalizzata e la pagina del singolo comune.

5.2 Struttura a componenti

L'interfaccia è suddivisa in componenti, ciascuno con un compito preciso:

- **Header** — barra superiore con marchio, ricerca dei comuni e selettore tema chiaro/scuro.
- **Striscia città preferite** — scorciatoie orizzontali alle principali città siciliane, con Palermo sempre in evidenza.
- **Meteo attuale** — scheda principale con temperatura, condizione e indicatori dei sensori.
- **Grafico orario** — andamento interattivo delle grandezze nell'arco della giornata.
- **Previsioni 7 giorni** — elenco giornaliero con icone e intervallo minima–massima.
- **Dettagli meteo** — insieme di schede di approfondimento in stile «app meteo».
- **Ricerca comune** — ricerca istantanea e *offline* tra i comuni siciliani.

Una **vista condivisa** (WeatherView) assembla questi componenti ed è usata sia dalla home sia dalla pagina di un comune scelto: così il comportamento è identico ovunque e il codice non viene duplicato.

Perché le chiamate ai dati esterni partono dal browser. I dati di Open-Meteo e le previsioni a 7 giorni vengono richiesti direttamente dal frontend. Questo alleggerisce il backend (che resta dedicato alla stazione), riduce i tempi di risposta e sfrutta un servizio gratuito senza chiavi d'accesso.

6. Gestione e integrazione dei dati — il caso Palermo

Una scelta centrale del progetto è che **Palermo non funziona come gli altri comuni**. Essendo la sede della stazione fisica, per Palermo i dati del momento provengono **direttamente dai sensori del Raspberry**, mentre la previsione delle ore successive è affidata al **modello di intelligenza artificiale** sviluppato internamente. Tutti gli altri comuni utilizzano invece i dati professionali di **Open-Meteo**.

6.1 Instradamento intelligente dei dati

La logica di scelta della fonte è incapsulata in alcuni **hook** dedicati (funzioni riutilizzabili di React) che decidono automaticamente da dove prendere i dati a seconda del comune richiesto:

- **Dati attuali** — per Palermo si apre il canale in tempo reale verso la stazione; per gli altri comuni si interroga Open-Meteo.
- **Serie oraria** — per Palermo si combinano lo storico reale dei sensori (passato e presente) e la previsione AI (futuro); per gli altri comuni si usa la previsione oraria di Open-Meteo.
- **Previsioni a 7 giorni** — sempre da Open-Meteo, anche per Palermo, perché sono previsioni multi-giorno che esulano dalla misura locale.

Distinzione tra misurato e previsto. Nel grafico la linea è **continua** per i dati realmente misurati e **tratteggiata** per quelli previsti (dal modello AI per Palermo, da Open-Meteo per gli altri). L'utente capisce immediatamente quali valori sono certi e quali sono stime.

Questa distinzione è coerente in **tutta** l'applicazione: ricerca, striscia delle città, grafico e schede di dettaglio trattano Palermo in modo speciale. Il modello di intelligenza artificiale verrà collegato all'endpoint già predisposto `/previsione/oggi`: fino ad allora, come soluzione transitoria, l'applicazione usa la previsione di Open-Meteo come riserva, garantendo che il sito sia comunque pienamente funzionante.

7. Il grafico orario interattivo in SVG

Il grafico dell'andamento orario è uno degli elementi più curati dell'applicazione. È stato disegnato interamente a mano in **SVG** (grafica vettoriale), senza ricorrere a librerie esterne di grafici.

Perché non una libreria pronta. Inizialmente era stata provata una libreria di grafici molto diffusa, che però risultava instabile con lo strumento di sviluppo usato, arrivando a rendere bianca l'intera pagina. Disegnare il grafico in SVG ci ha dato **controllo totale** sull'aspetto, prestazioni migliori e perfetta resa anche su smartphone, eliminando la dipendenza problematica.

7.1 Cosa permette di fare

- **Nove grandezze selezionabili** tramite schede: temperatura, precipitazioni, vento, qualità dell'aria, umidità, nuvolosità, pressione, luce e monossido di carbonio.
- **Navigazione tra i giorni**, da ieri fino a sei giorni avanti, per leggere l'andamento di ciascuna giornata.
- **Tooltip interattivo** che, al passaggio del dito o del mouse, mostra ora precisa, valore e icona del tempo.
- **Indicatori di alba e tramonto** e una marcatura dell'ora corrente con etichetta «Adesso».
- **Curva morbida** ottenuta con un'interpolazione matematica (Catmull-Rom convertita in curve di Bézier) per un tratto elegante e naturale.

- **Scorrimento orizzontale** con centratura automatica sull'ora corrente: comodo soprattutto su telefono.

Il grafico è protetto da un meccanismo di sicurezza (*error boundary*) che, in caso di anomalia, mostra un messaggio discreto invece di compromettere l'intera pagina.

8. I «Dettagli meteo»

Sotto il grafico, una sezione di **schede di approfondimento** — ispirata nello stile alle grandi app meteo ma costruita sui **nostri dati** — riassume in modo visivo ogni grandezza. Ogni scheda combina un valore numerico, una breve interpretazione testuale e una piccola grafica vettoriale.

8.1 Le schede e i valori derivati

Oltre ai valori grezzi dei sensori, l'applicazione calcola alcune grandezze **derivate** che arricchiscono la lettura:

- **Temperatura percepita** — combina temperatura, umidità e vento (indice di calore o *wind chill*).
- **Punto di rugiada** — calcolato con la formula di Magnus a partire da temperatura e umidità.
- **Scala Beaufort** — traduce la velocità del vento in una forza con descrizione testuale (es. «Brezza tesa»).
- **Stato del cielo e qualità dell'aria** — percentuali tradotte in etichette e colori immediati.
- **Tendenze** — indicazioni come «in aumento»/«in calo» per temperatura e pressione, e l'ora del prossimo picco.

Tutte le micro-grafiche (curve, archi, cerchi di riempimento) sono realizzate in **SVG inline**: leggere, nitide su ogni schermo e senza librerie esterne.

9. Design, layout e adattamento responsive

L'interfaccia è stata progettata per essere **responsive**, cioè per adattarsi automaticamente alle dimensioni dello schermo. Il layout di base si ispira alle moderne app meteo: striscia delle città in alto, meteo attuale e grafico al centro, previsioni a sette giorni di fianco e schede di dettaglio in basso.

9.1 Tre formati, una sola esperienza

- **Computer e tablet** — disposizione su più colonne (fino a tre) per sfruttare lo spazio: meteo attuale e grafico affiancati alle previsioni, dettagli a tre colonne.
- **Smartphone (verticale)** — tutto si riorganizza in una colonna ordinata e leggibile.

9.2 L'ottimizzazione per telefono

La versione per smartphone ha richiesto un lavoro specifico per renderla, oltre che funzionale, anche **esteticamente curata**. Gli interventi principali:

- **Intestazione «hero» centrata** — su telefono la scheda principale presenta in primo piano, al centro, la temperatura grande e l'indicazione della qualità dell'aria.
- **Niente dati duplicati** — le sei metriche che su desktop affiancano la temperatura vengono nascoste sul telefono, perché già presenti, in forma più ricca, nelle schede di dettaglio: meno ripetizioni, meno scorrimento.
- **Dettagli a due colonne** — le schede si dispongono a due colonne sui telefoni normali e passano automaticamente a una sola colonna sugli schermi più stretti, così i numeri non vengono mai tagliati.
- **Spaziature e titoli adattivi** — margini ridotti e testi che vanno a capo correttamente, senza fuoriuscire dai bordi.

- **Protezione dallo straripamento orizzontale** per evitare che il contenuto risulti compresso.

Tema chiaro e scuro. L'applicazione supporta entrambi i temi; i colori sono definiti con lo spazio colore moderno `oklch`, che garantisce tinte uniformi e gradevoli in ogni modalità. L'estetica adotta uno stile «vetro» (*glassmorphism*): schede semitrasparenti con leggera sfocatura dello sfondo.

10. Identità visiva: logo e palette

Per dare al progetto un'identità riconoscibile è stato creato un **logo originale**:

- **Marchio testuale** — la scritta «Meteo Sicilia», con «Sicilia» in gradiente, e il sottotitolo «Meteo in tempo reale».

10.1 Nome a dominio consigliato

Per la pubblicazione è consigliabile un dominio breve, facile da ricordare e coerente col nome del progetto (ad esempio nella forma `meteosicilia` con estensione `.com` oppure `.it`), evitando nomi troppo lunghi o complicati che renderebbero difficile trovare il sito.

11. Scelte tecniche e motivazioni

Riepilogo ragionato delle decisioni principali e del loro **perché**.

Scelta	Motivazione
MQTT per la stazione	Standard leggero e affidabile per l'IoT; disaccoppia stazione e server e apre alla futura aggiunta di più stazioni.
FastAPI (Python, asincrono)	Veloce, adatto a flussi in tempo reale (SSE) e a molte connessioni simultanee; codice chiaro e moderno.
SQLite	Database leggero, senza server dedicato: ideale per una stazione locale.
SSE per il tempo reale	Standard web semplice ed efficiente per «spingere» aggiornamenti al sito senza ricaricare.
React + TypeScript	Interfaccia a componenti riutilizzabili e controllo dei tipi che previene molti errori.
Tailwind CSS	Impaginazione rapida, coerente e particolarmente comoda per il design responsive.
Grafico in SVG (no librerie)	Controllo totale, prestazioni e stabilità; risolve i problemi avuti con una libreria esterna.

Open-Meteo per gli altri comuni	Servizio meteo professionale e gratuito, senza chiavi d'accesso, interrogabile dal browser.
Interventi backend additivi	Non stravolgere il lavoro esistente sui sensori: massima compatibilità e affidabilità.

12. Problemi affrontati e soluzioni

Lo sviluppo ha richiesto di individuare e risolvere diverse criticità. Le principali:

Problema	Soluzione adottata
Codice ereditato da tutorial, con riferimenti a funzioni e dati inesistenti, che impediva la compilazione.	Pulizia e riscrittura dei moduli interessati, eliminando le parti incoerenti e ricostruendo le pagine sulla nostra architettura.
La libreria di grafici rendeva bianca la pagina con lo strumento di sviluppo usato.	Rimozione della libreria e realizzazione del grafico interamente in SVG.
L'API non rispondeva alle chiamate del sito (blocco cross-origin del browser).	Configurazione del middleware CORS lato backend, in modo additivo.
Necessità di trattare Palermo diversamente dagli altri comuni.	Hook di instradamento che scelgono automaticamente la fonte (stazione+AI per Palermo, Open-Meteo altrove) in modo coerente in tutta l'app.

13. Sviluppi futuri

- **Integrazione del modello di intelligenza artificiale** sull'endpoint già predisposto, per la previsione delle ore successive a Palermo.
- **Più stazioni** in altre città siciliane, sfruttando la struttura già pensata per essere estesa.
- **Notifiche e installazione come app** (PWA), per avvisi meteo e accesso rapido dal telefono.
- **Archiviazione storica e statistiche** (medie, record, confronti tra periodi) a partire dai dati raccolti dai sensori.
- **Valorizzazione delle immagini** della fotocamera, ad esempio una galleria del cielo sincronizzata con i dati.

14. Conclusioni

Il progetto **Meteo Sicilia** realizza un sistema completo che parte dal **dato fisico misurato** da una stazione costruita con Raspberry Pi e arriva fino alla sua **rappresentazione curata** su un'applicazione web moderna, funzionante su qualsiasi dispositivo. Sono stati affrontati e integrati ambiti diversi — elettronica e sensori, protocolli IoT, programmazione lato server in tempo reale, sviluppo web a componenti, design e sicurezza — in un'unica soluzione coerente.

In sintesi. Meteo Sicilia non è «solo un sito meteo»: è una catena completa che collega un sensore reale a uno schermo, con scelte tecniche giustificate, un'identità visiva propria e un'esperienza d'uso curata su computer, tablet e smartphone.

Relazione tecnica — Meteo Sicilia